EL APPLICATIONS WEB DISTRIBUÉES · MT-41

Arène

Plateforme de hackathons & bounties

REVUE DE CADRAGE ET D'ARCHITECTURE · SÉANCE 4, JALON 1

VALIDATION FINALE · 2026-2027

Ce que nous validons aujourd'hui

01 **Cadrage**

Idée, besoin, utilisateurs, scénarios, périmètre MoSCoW.

02 **Architecture**

Diagramme global, six microservices, stratégie de données.

03 **Communication**

Synchrone (Feign) et asynchrone (RabbitMQ), Gateway et sécurité.

04 **Exécution**

Contrats d'API, stack, répartition des tâches, planning, risques.

L’idée et le besoin

Le problème

- Une entreprise a un **besoin logiciel ponctuel**.
- Elle ne veut ni **recruter une équipe**, ni payer des freelances au forfait.
- Les développeurs veulent des **projets réels**, un **prix** et de la **visibilité**.

La solution : Arène

- L’entreprise publie un **défi** : spec, **tests d’acceptation** et **prime (top 3)**.
- Les développeurs soumettent leur **dépôt GitHub**.
- Un **moteur de vérification** rejette automatiquement les échecs.
- Les **experts** évaluent et désignent les **gagnants**.

Valeur métier : réduire le coût d’accès à du logiciel sur mesure, et offrir aux développeurs un portfolio *vérifiable*.

Les utilisateurs

Entreprise

Obtenir un logiciel conforme, à coût maîtrisé, avec une qualité garantie.

Développeur / Équipe

Projets réels, prime, visibilité et profil valorisant pour l'embauche.

Expert

Évaluer les projets sur des critères clairs et pondérés.

Admin

Modérer, gérer les litiges et les comptes.

Rôles Keycloak : `DEVELOPER` `ENTERPRISE` `EXPERT` `ADMIN`

Chaque route est protégée par rôle (`@PreAuthorize`) : l'entreprise publie et finance, le développeur soumet, l'expert évalue, l'administrateur modère.

Scénario de bout en bout

01

L'entreprise publie un défi : spec, tests, primes, date limite.

02

Le développeur s'inscrit, seul ou en équipe.

03

Il soumet son dépôt GitHub, avec un commit figé.

04

Le moteur exécute les tests et rend un verdict.

05

Les experts notent les projets retenus.

06

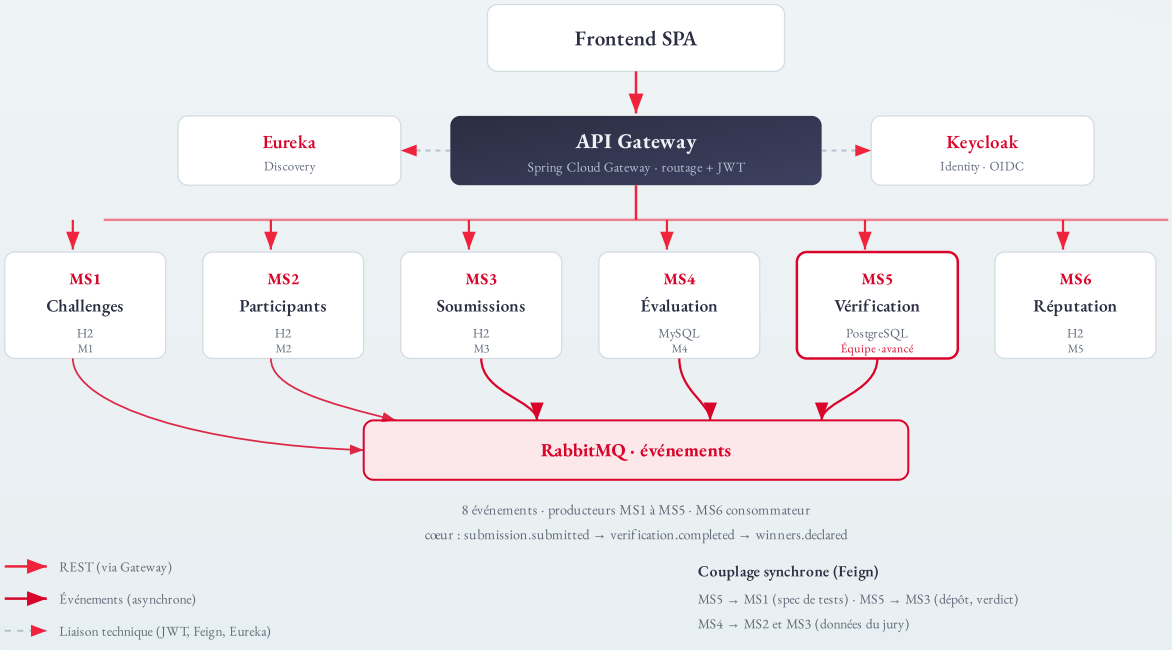
Les gagnants (top 3) sont désignés et récompensés.

Un FAIL aux tests d'acceptation déclenche un **rejet automatique**. Les logs restent consultables par l'équipe.

Priorisation MoSCoW

Must	Publier un défi · s'inscrire · soumettre un dépôt · **exécuter les tests** · verdict PASS/FAIL et rejet automatique · notation · classement · gagnants · auth Keycloak · Gateway et Eureka.
Should	Réputation et badges · feedback du jury · tableau de bord entreprise · notifications d'événements · anti-plagiat.
Could	Appariement experts et défis · résumé IA des soumissions · rédaction assistée des tests · messagerie · statistiques.
Won't	Paiement réel (simulé) · dépôts GitHub privés · exécution multi-langages · application mobile.

Architecture globale



Décomposition en microservices

Cinq services portés individuellement, plus un sixième porté par l'équipe sur une technologie avancée.

MS1 M1

Challenges

Espace entreprise, défis (spec, critères, tags, dates, cycle de vie), budget et primes, catalogue.

`H2`

MS2 M2

Participants

Développeurs, équipes, inscriptions et candidatures aux défis.

`H2`

MS3 M3

Soumissions

Dépôt GitHub, commit figé, artefacts, cycle de vie, contrôle de la date limite.

`H2`

MS4 M4

Évaluation

Experts, jury, grilles pondérées, scores, leaderboard, désignation des gagnants.

`MySQL`

MS5 ÉQUIPE

Vérification AVANCÉ

Technologie avancée : tests en bac à sable, anti-plagiat, IA d'appoint.

`PostgreSQL + pgvector`

MS6 M5

Réputation

Profils, réalisations vérifiées, compétences, classements.

`H2`

Infrastructure partagée (équipe) : **API Gateway**, **Keycloak**, **Eureka**, **Docker/Compose**, **Frontend**.

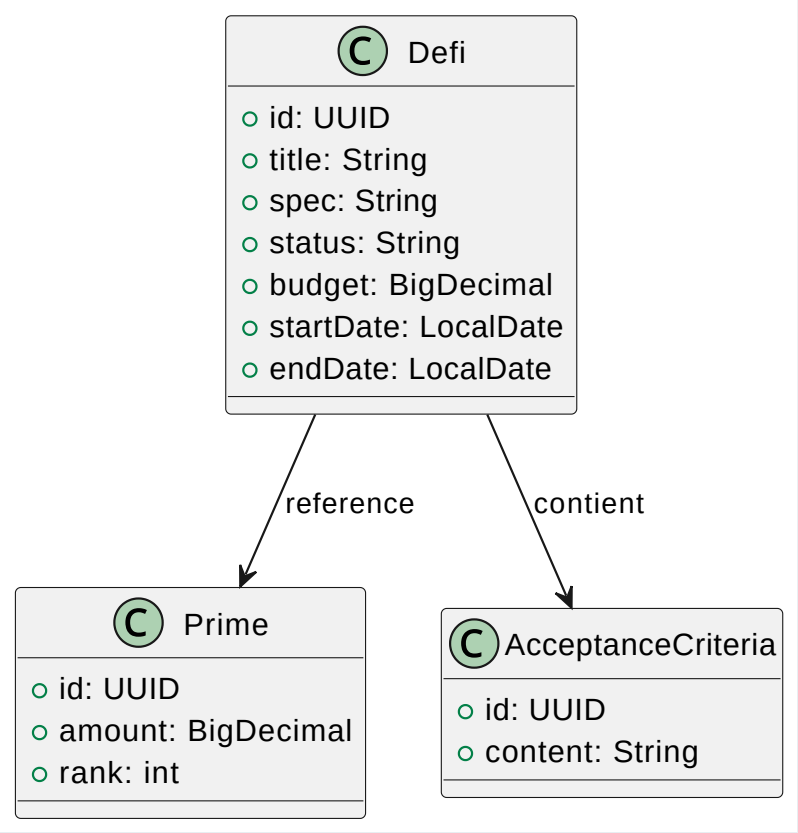
MS1 · Challenges

Publication du défi : cahier des charges, critères d'acceptation, primes et catalogue.

PROPRIÉTAIRE	M1
BASE DE DONNÉES	H2

MS1 · Challenges

POST	/api/challenges
GET	/api/challenges
GET	/api/challenges/{id}
PUT	/api/challenges/{id}
POST	/api/challenges/{id}/publish
GET·PUT	/api/challenges/{id}/acceptance-criteria
GET·PUT	/api/challenges/{id}/prizes



MS1: Responsabilité : publication du défi, cahier des charges, critères d'acceptation, primes et catalogue.

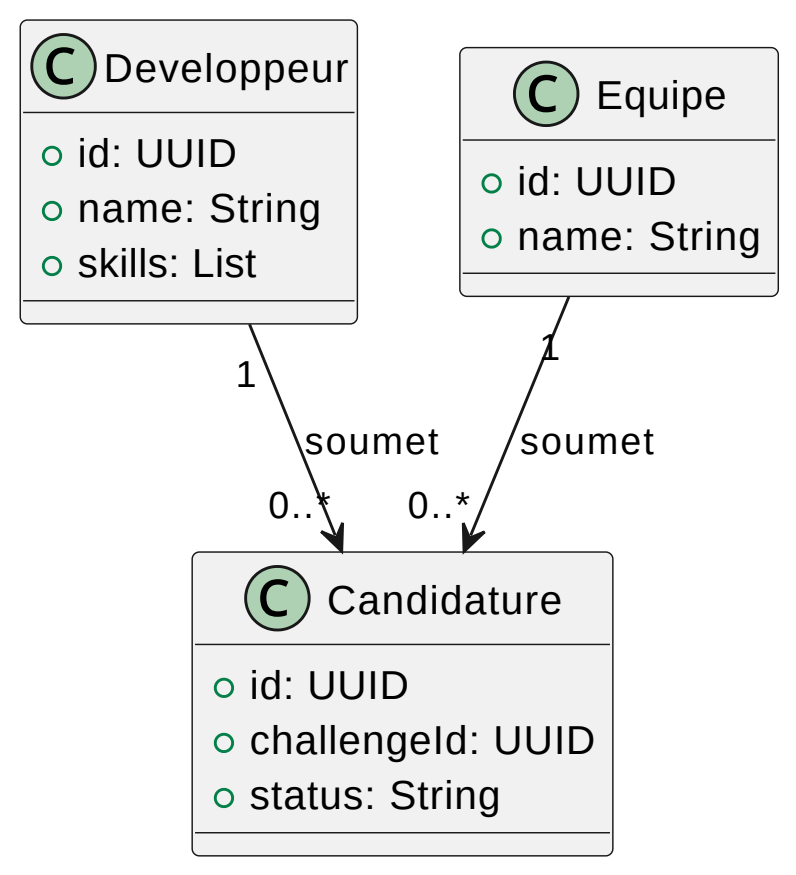
MS2 · Participants

Développeurs et équipes : profils, inscriptions et candidatures aux défis.

PROPRIÉTAIRE	M2
BASE DE DONNÉES	H2

MS2 · Participants

POST	/api/developers
GET	/api/developers/{id}
POST	/api/teams
GET	/api/teams/{id}
POST·GET	/api/challenges/{challengeId}/applications



MS2: Responsabilité : profils de développeurs, équipes et candidatures.

MS3 · Soumissions

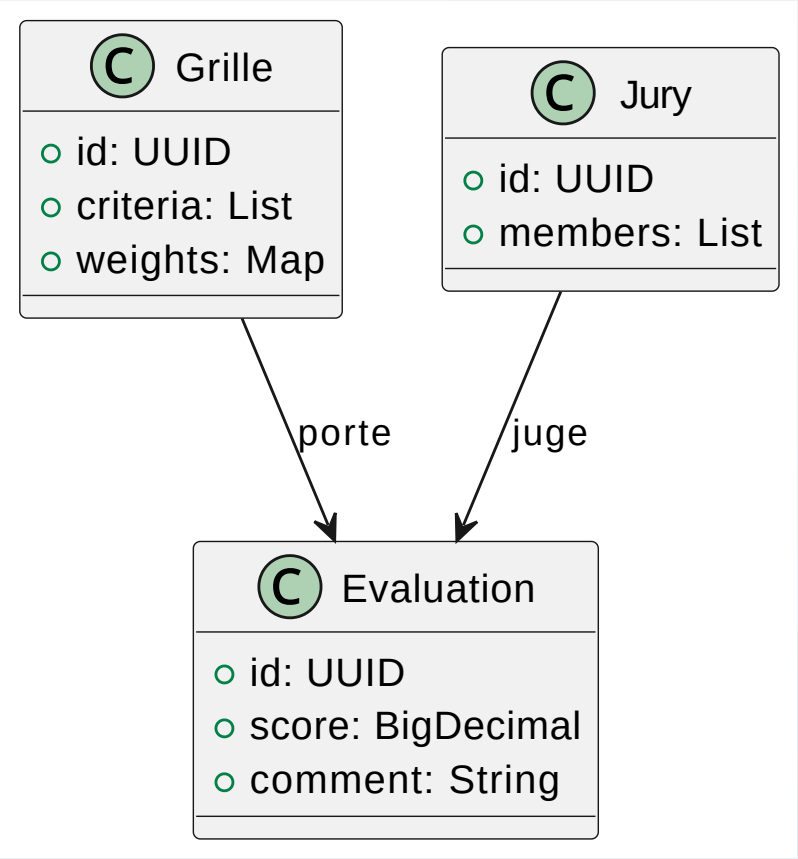
MS4 · Évaluation

Jury, grilles pondérées, scores, classement pondéré, désignation des gagnants.

PROPRIÉTAIRE	M4
BASE DE DONNÉES	MySQL

MS4 · Évaluation

POST	/api/experts
POST·GET	/api/challenges/{challengeId}/jury
POST	/api/rubrics
GET	/api/rubrics
POST	/api/evaluations
GET	/api/challenges/{challengeId}/leaderboard
POST	/api/challenges/{challengeId}/declare-winners



MS4: Responsabilité : jury, grilles pondérées, scores, classement et désignation des gagnants.

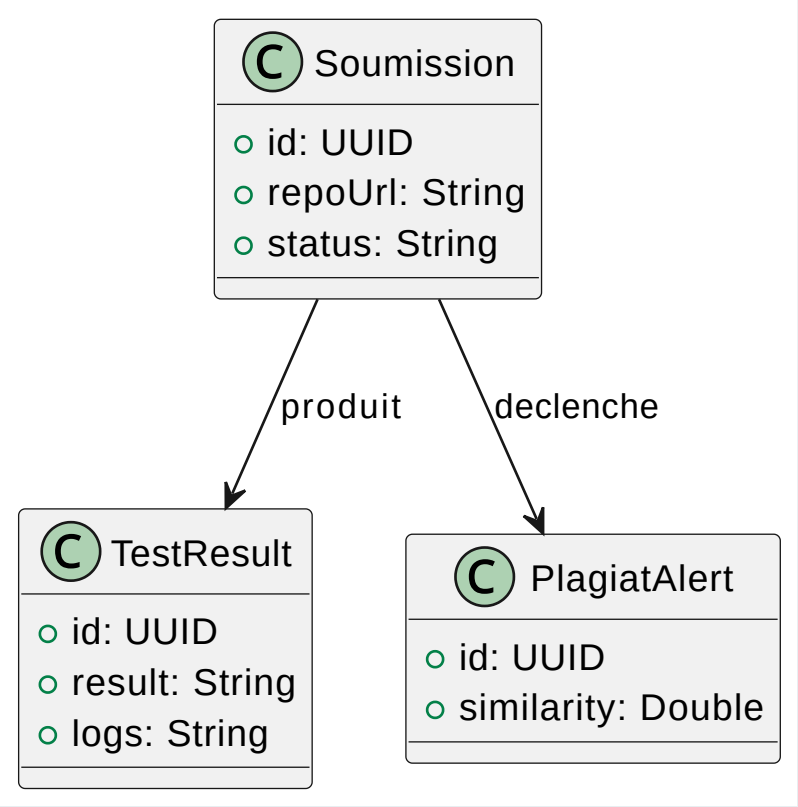
MS5 · Vérification

Technologie avancée : tests en bac à sable, anti-plagiat, IA d'appoint.

PROPRIÉTAIRE	Équipe
BASE DE DONNÉES	`PostgreSQL + pgvector`

MS5 · Vérification

POST	/api/verifications
GET	/api/verifications/{id}
GET	/api/verifications/{id}/logs
POST	/api/similarity/analyze
GET	/api/similarity/{submissionId}
POST	/api/ai/summarize/{submissionId}



Vérification · service avancé: Bac à sable, anti-plagiat et IA d'appoint : déterministe au cœur, IA optionnelle.

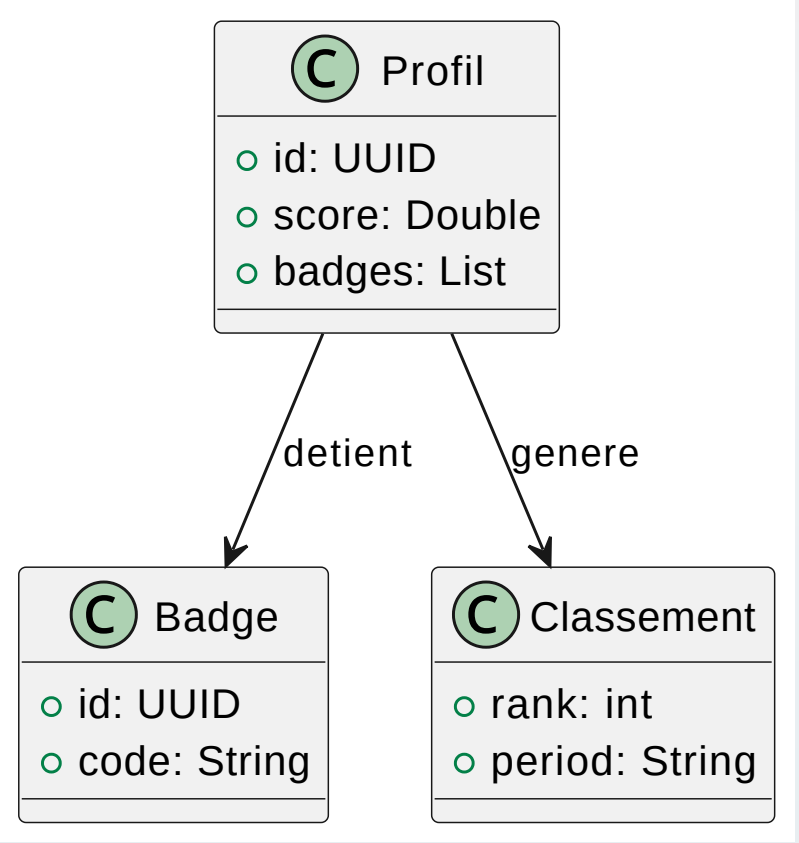
MS6 · Réputation

Profils de réputation, badges et classement.

PROPRIÉTAIRE	M5
BASE DE DONNÉES	H2

MS6 · Réputation

GET	/api/developers/{id}/reputation
GET	/api/developers/{id}/badges
GET	/api/rankings
POST	/api/reputation/events



MS6: Responsabilité : réputation, badges et classements.

Une base par service

SERVICE	BASE	JUSTIFICATION
MS1 · Challenges	`H2`	Données relationnelles simples (défis, primes)
MS2 · Participants	`H2`	Entités légères (développeurs, équipes, candidatures)
MS3 · Soumissions	`H2`	Métadonnées de dépôts, cycle de vie
MS4 · Évaluation	`MySQL`	Scores, grilles et agrégations, donc un SGBD robuste
MS5 · Vérification	`PostgreSQL + pgvector`	Résultats de tests (JSONB) et similarité vectorielle
MS6 · Réputation	`H2`	Profils, badges, classements

> **Règle d'or :** une base par service, *aucune clé étrangère inter-services*. Les références croisées passent par identifiant et par événements.

Pas de transaction distribuée : la cohérence entre services est **à terme**, portée par les événements, et chaque service migre son propre schéma.

Communication synchrone

Frontend	→	**Gateway**

Routage et validation du JWT vers tous les services.

MS5 · Vérification	→	**MS1 · Challenges**

Récupérer la spec de tests d'acceptation du défi.

MS5 · Vérification	⇄	**MS3 · Soumissions**

Récupérer le dépôt de la soumission, puis pousser le verdict.

MS4 · Évaluation	→	**MS3 et MS2**

Données de soumission et de participants pour le jury.

> **REST via OpenFeign.** Appels directs via **Eureka**, hors Gateway. Relais du jeton pour les appels utilisateur, compte de service pour les traitements d'événement. Résilience : délais courts, réessais bornés, disjoncteur (`Resilience4j`).

Communication asynchrone

RabbitMQ via Spring Cloud Stream : le flux cœur est entièrement événementiel.

ÉVÉNEMENT	PRODUCTEUR	CONSOMMATEURS
`challenge.published`	**MS1**	MS2, MS4
`application.registered`	**MS2**	MS1, MS3
`submission.submitted`	**MS3**	MS5, MS4
`verification.completed`	**MS5**	MS3, MS4, MS2, MS6
`submission.rejected`	**MS5**	MS3, MS2
`judging.completed`	**MS4**	MS1, MS2
`winners.declared`	**MS4**	MS1, MS2, MS6
`similarity.flagged`	**MS5**	MS4, MS3

Enveloppe commune : `eventId`, `type`, `occurredAt`, `producer`, `version`, `payload`. Un groupe durable par consommateur, une file de rebut en cas d'échec, des consommateurs **idempotents** par `eventId` : livraison « au moins une fois ».

MS5 : la technologie avancée

Moteur de tests

Déclenché par `submission.submitted`, il récupère le dépôt et exécute les tests dans un **conteneur isolé**.

Verdict : `{PASS|FAIL, tests, couverture, logs}`

Isolation : `--network none`, mémoire et CPU plafonnés, utilisateur non-root, système de fichiers en lecture seule, délai maximal strict.

Anti-plagiat

Embeddings des soumissions et similarité cosinus via **pgvector** : alerte le jury avant la notation.

IA d'appoint

Résumé de soumission pour le jury, rédaction de tests à partir des exigences, appariement experts et défis.

L'IA est une amélioration, pas le cœur. `docker compose stop ms5-verification` : la plateforme reste utilisable en vérification manuelle ; seuls les badges IA et l'anti-plagiat disparaissent.

Gateway, sécurité et configuration

API Gateway

RÔLE	Point d'entrée unique, routage vers les services via Eureka.
SÉCURITÉ	Validation des JWT Keycloak, CORS, rate limiting léger.

Keycloak

REALM	`arena` et rôles DEVELOPER, ENTERPRISE, EXPERT, ADMIN.
CLIENTS	Frontend public (Authorization Code + PKCE), gateway confidentiel.

L'authentification relève de Keycloak (infrastructure), pas d'un microservice. On le configure, on ne le code pas : chaque service valide simplement le JWT (`SecurityFilterChain` , `@PreAuthorize`). Un service d'auth maison ne serait qu'un habillage, donc un anti-pattern.

Configuration et conventions. Un `Dockerfile` par service et un `docker-compose` de bout en bout, profils `dev` et `docker` , secrets par variables d'environnement. Erreurs `ErrorResponse` , pagination `page/size/sort` , versionnage `/api/v2` , idempotence par `eventId` .

Contrats d'API : cœur

MS1 · Challenges

`POST`	`/api/challenges`
`GET`	`/api/challenges`
`GET`	`/api/challenges/{id}`
`PUT`	`/api/challenges/{id}`
`POST`	`/api/challenges/{id}/publish`
`GET·PUT`	`/api/challenges/{id}/acceptance-criteria`
`GET·PUT`	`/api/challenges/{id}/prizes`

MS2 · Participants

`POST`	`/api/developers`
`GET`	`/api/developers/{id}`
`POST`	`/api/teams`
`GET`	`/api/teams/{id}`
`POST·GET`	`/api/challenges/{challengeId}/applications`

MS3 · Soumissions

`POST`	`/api/submissions`
`GET`	`/api/submissions`
`GET`	`/api/submissions/{id}`
`POST`	`/api/submissions/{id}/submit`
`GET·PUT`	`/api/submissions/{id}/verdict`

Documentation **Swagger / OpenAPI** générée par `springdoc-openapi` sur chaque service. Les paramètres de chemin sont notés entre accolades.

Contrats d'API : jury et avancé

MS4 · Évaluation

POST	/api/experts
POST·GET	/api/challenges/{challengeId}/jury
POST	/api/rubrics
GET	/api/rubrics
POST	/api/evaluations
GET	/api/challenges/{challengeId}/leaderboard
POST	/api/challenges/{challengeId}/declare-winners

MS5 · Vérification

POST	/api/verifications
GET	/api/verifications/{id}
GET	/api/verifications/{id}/logs
POST	/api/similarity/analyze
GET	/api/similarity/{submissionId}
POST	/api/ai/summarize/{submissionId}

MS6 · Réputation

GET	/api/developers/{id}/reputation
GET	/api/developers/{id}/badges
GET	/api/rankings
POST	/api/reputation/events

Les méthodes partageant une même ressource sont groupées sur une seule ligne (`GET·PUT`, `POST·GET`).

Stack technologique

LANGAGE	**Java et Spring Boot**, standard des microservices, imposé par le cours.
GATEWAY	**Spring Cloud Gateway** : routage, filtres, intégration Spring Security.
DÉCOUVERTE	**Eureka** : enregistrement, découverte, équilibrage côté client.
SÉCURITÉ	**Keycloak** : OIDC et JWT, rôles, standard éprouvé.
OBSERVABILITÉ	**Spring Boot Actuator** (`/health`), consoles Eureka et RabbitMQ, journaux corrélés par `eventId`.
COMMUNICATION	**OpenFeign et RabbitMQ** : couvre les deux modes exigés.
BASES	**H2, MySQL, PostgreSQL et pgvector** : persistance par service, pgvector pour la similarité.
CONTENEURS	**Docker et Compose** : lancement reproductible de bout en bout.
FRONTEND	**Angular 21 (Node 22)** : SPA consommant les API via la Gateway.

Répartition des tâches

M1	**MS1 · Challenges**	API défis, spec, critères, primes, catalogue, tests
M2	**MS2 · Participants**	API développeurs et équipes, inscriptions, validations
M3	**MS3 · Soumissions**	API soumissions, dépôt GitHub, cycle de vie, date limite
M4	**MS4 · Évaluation**	API jury, grilles, scores, leaderboard, gagnants
M5	**MS6 · Réputation**	API profils, badges, classements
Équipe	**MS5 + Infra + Front**	Bac à sable de tests, anti-plagiat, IA · Gateway, Keycloak, Eureka, Docker · écrans

Intégration : contrats gelés en S4, développement indépendant sur mocks, intégration S5-S9, démonstration S10.

Gestion de projet et organisation Git

GitHub Projects

Backlog	À faire	En cours	Revue	Fait

- Un **label par microservice**.
- Une **milestone par séance**, de S4 à S10.
- Epics *Jalon 1* et *Jalon 2*.

Git

```
main ← develop ← feature/msX-…
```

- PR avec revue avant merge.
- Commits conventionnels : `feat(ms1): …`.
- Commits réguliers et **traçables par membre**, pour l'évaluation individuelle.
- Intégration continue (optionnelle) : build et tests à chaque PR.

Feuille de route et points de vigilance

FEUILLE DE ROUTE

S4	Validation de conception (ce document)
S5	Gateway, Eureka et Compose local
S6	Communication synchrone (Feign)
S7	Asynchrone (RabbitMQ) et pipeline de vérification
S8	Sécurité Keycloak (realm, rôles, JWT)
S9	Dockerisation complète et bac à sable
S10	Démonstration et soutenance

RISQUES ET MITIGATION

ÉLEVÉ	Exécution de code non fiable : conteneurs isolés (réseau coupé, mémoire et CPU limités, non-root, timeout). Variante : image Docker et commande fournies.
MOYEN	Périmètre large : priorisation MoSCoW, le catalogue d'événements borne les responsabilités.
MOYEN	Intégration tardive : contrats gelés en S4 et mocks WireMock.
MOYEN	Démo dépendante d'API externes : jeux de données pré-remplis, profils mock.

EL MERCI DE VOTRE ATTENTION

Merci

Entrez en arène.

PROCHAINES ÉTAPES · SQUELETTES SPRING BOOT, DOCKER-COMPOSE ET MAQUETTES FRONTEND
LIVRABLES S4 · CE DOSSIER, LE DIAGRAMME D'ARCHITECTURE ET LES CONTRATS D'API
PLACE À LA DISCUSSION.